VERIQTA

INCIDENT RESPONSE AND DFIR

DETECT | INVESTIGATE | CONTAIN | RECOVER | LEARN



DETECT THREATS | **INVESTIGATE INCIDENTS** | **CONTAIN BREACHES** | **RECOVER SECURELY** | **LEARN AND IMPROVE**

@veriqta_academy | @veriqta_academy | @veriqta_academy | @veriqta_academy | @veriqta_academy | @veriqta_academy

Incident Response and DFIR

What is Incident Response?

- Incident response is a structured process to **detect, investigate, contain, recover and learn** from security incidents.
- Its goal is to reduce the impact of an attack and get systems back to a safe and normal state.
- Incident response is used when something bad has happened or is happening.

What is DFIR?

- DFIR stands for **Digital Forensics and Incident Response.**
- It combines incident response with forensic investigation.
- Digital forensics focuses on collecting, preserving and analyzing digital evidence.
- DFIR helps answer questions like:
  - What happened?
  - How did it happen?
  - Who did it?
  - What was affected?
  - How can we prevent it from happening again?

Security Event vs Security Incident

Security Event	Security Incident
• A single activity or alert that may or may not be malicious.	• A confirmed or highly likely security event that causes harm or has the potential to cause harm.
• Can be normal or suspicious.	• Requires investigation and response.
• Example: a **failed SSH login attempt.**	• Example: **multiple failed logins followed by a successful unauthorized login.**

The Junior Engineer's Role



- Monitor alerts and escalate suspicious activity.
- Collect and preserve evidence as instructed.
- Run initial investigation commands.
- Document findings clearly.
- Follow the incident response process.
- Avoid making changes that can destroy evidence.
- Work with the security team and follow guidance.

Incident Response Flow

DETECT
- Identify suspicious activity through alerts, logs or monitoring tools.

INVESTIGATE
- Confirm what happened, how it happened and what was affected.

CONTAIN
- Stop the attack from spreading and reduce further damage.

RECOVER
- Restore systems to a safe and normal state.

LEARN
- Find the root cause and improve processes to prevent it in the future.

Key Takeaway

Incident response and DFIR help organizations detect attacks early, understand what happened, contain the impact, recover safely and prevent it from happening again. As a junior engineer, your role is to observe, investigate, preserve evidence, follow the process and escalate when needed.

The Incident Response Lifecycle

1. PREPARATION — Be ready before an incident happens.

2. DETECTION AND ANALYSIS — Identify and understand what is happening.

3. CONTAINMENT — Stop the incident from spreading and reduce impact.

4. ERADICATION — Remove the root cause and malicious artifacts.

5. RECOVERY — Restore systems to a safe and normal state.

6. POST-INCIDENT LEARNING — Find lessons, improve defenses and prevent it happening again.

Continuous Investigation and Evidence Collection

1. PREPARATION

- Put security controls in place before an incident happens.
- Define roles and responsibilities.
- Enable logging and monitoring.
- Prepare tools and access.
- Create runbooks and communication plans.
- Train the team and run simulations.

2. DETECTION AND ANALYSIS

- Detect suspicious activity through alerts, logs or monitoring tools.
- Confirm if it is a true incident.
- Understand what happened, how it happened and the potential impact.
- Identify affected systems, users and data.
- Collect initial evidence.

3. CONTAINMENT

- Stop the incident from spreading to other systems.
- Isolate affected systems when needed.
- Disable compromised accounts or credentials.
- Block malicious network traffic.
- Containment can be short-term or long-term.

4. ERADICATION

- Remove the root cause of the incident.
- Delete malicious files, tools and backdoors.
- Patch exploited vulnerabilities.
- Remove unauthorized access and accounts.
- Ensure the attacker can no longer return.

5. RECOVERY

- Restore systems and services to a safe and normal state.
- Validate that the environment is clean and stable.
- Monitor closely for signs of recurrence.
- Return to normal operations in a controlled and verified manner.

6. POST-INCIDENT LEARNING

- Review what happened.
- Identify what worked well and what did not.
- Improve detection, response and prevention.
- Update security controls, runbooks and documentation.
- Share lessons with the team.
- Run tabletop exercises based on real incidents.

Why Investigation Continues Throughout the Lifecycle

- Investigation does not stop after detection. It continues in every phase.
- You keep collecting and analyzing evidence to understand the full scope, remove the attacker completely, confirm the environment is clean and identify lessons for the future.
- Even during containment, eradication and recovery, you may find new evidence that changes what happened or reveals other affected systems.
- Continuous investigation helps prevent reinfection and ensures nothing is missed.

Is This Actually an Incident?

Event vs Alert vs Incident

EVENT

- A single activity that happens in a system.
- Can be normal or suspicious.
- Examples: user login, service start, file change.

ALERT

- A notification that something may be wrong.
- Generated by monitoring tools or security rules.
- Needs investigation to confirm.
- Example: multiple failed login attempts.

INCIDENT

- A confirmed or highly likely event that causes harm or has the potential to cause harm.
- Requires investigation and response.
- Examples: unauthorized access, malware infection, data exposure.

Examples of Normal vs Suspicious Behavior

Activity	Normal Behavior	Suspicious Behavior
User Login	Your usual login from known location	Login from unknown country or at unusual time
Failed Logins	A few failed attempts	Many failed attempts in a short time
Process	Expected system process running	Unknown or unusual process running
Network Traffic	Normal internal or application traffic	Unexpected outbound traffic to unknown IP
File Change	Expected config update by admin	Important system files changed without approval
Service	Service running as expected	Critical service stopped or restarted unexpectedly

Failed SSH Attempts

- A few failed attempts can be normal.
- Many failed attempts may indicate a brute force attack.
- Check the source IP, usernames and pattern.
- If followed by a successful login, treat it as suspicious and investigate.

Unexpected Process

- Look for processes you do not recognize.
- Check the process name, command line, user and parent process.
- It could be malware, a reverse shell or an unauthorized tool.
- Verify if it is expected before taking action.

Unusual Outbound Traffic

- Most servers should not initiate random outbound connections.
- Unusual traffic to unknown IP addresses or domains can indicate malware or data exfiltration.
- Check the destination, port, process and frequency.
- Compare with normal application behavior.

Unauthorized Configuration Change

- Changes to critical files without approval can indicate compromise.
- Examples: SSH config, firewall rules, user accounts, scheduled tasks.
- Check who made the change, when it happened and what was modified.
- Unexpected changes require investigation.

When to Escalate as a Junior Engineer



- You see signs of unauthorized access.
- You find malware or a suspicious process.
- There is unusual outbound traffic.
- Important system files were changed.
- A critical service is affected.
- You are not sure if something is malicious.
- The incident may involve sensitive data.
- When in doubt, escalate. It is better to ask early than to miss a real incident.

Key Takeaway

- Not every alert is an incident.
- Investigate to understand what is really happening.
- Look for patterns, not a single event.
- Compare with normal behavior.
- Use evidence, not assumptions.
- Know when to escalate.
- Your job as a junior engineer is to observe, investigate and report early.

Severity and Incident Triage

What Does Triage Mean?



- Triage is the process of quickly assessing an incident to understand its impact, urgency and required response.
- It helps you prioritize work and focus on the most critical issues first.
- You collect key information, categorize the incident and decide the next steps.

Impact and Urgency



- **Impact:** How much damage the incident causes. Examples: system downtime, data loss, financial loss, customer impact.
- **Urgency:** How quickly action is needed. Examples: active attack, spreading malware, service completely down.
- Both impact and urgency determine the severity level.

SEV1 to SEV4 Severity Model

Severity	Description	Typical Impact	Examples
SEV1	Critical (Immediate action)	• Major service outage • Large number of users affected • Significant data loss or security breach	• Production system down • Active ransomware attack • Major data exposure
SEV2	High (Urgent)	• Partial service outage • Limited number of users affected • Potential data exposure	• Key service degraded • Suspicious activity in production • Unauthorized access detected
SEV3	Medium (Important)	• Minor service issue • Small number of users affected • No confirmed data exposure	• Non-production compromised • Repeated failed login attempts • Suspicious process detected
SEV4	Low (Monitor and fix)	• Minimal impact • No user impact or very limited • No data exposure	• Single failed login attempt • Policy violation • Informational security event

Systems Affected



- Which systems are impacted?
- Production or non-production?
- Single server or multiple systems?
- Is it a core business system?

Users Affected



- How many users are affected?
- Internal users, customers or both?
- Are critical teams impacted?
- Is the issue widespread or isolated?

Data Exposure



- Has sensitive data been accessed?
- What type of data? (e.g. customer data, financial data, source code)
- Is the data confirmed or suspected to be exposed?

Business Impact



- Does it affect revenue?
- Does it impact customers?
- Is there a legal or compliance risk?
- Does it damage the company's reputation?

Why Severity Can Change During Investigation



- You may start with limited information.
- As you investigate, you can discover a larger impact or data exposure.
- A low-severity alert may turn into a high-severity incident.
- New evidence, affected systems or user impact can increase or decrease the severity level.
- Always keep reviewing and update the severity as new facts emerge.

Key Takeaway



- Triage helps you focus on what matters most.
- Severity depends on impact and urgency.
- Consider systems, users, data and business impact.
- Severity can change as you learn more.
- Accurate triage ensures the right response and quicker resolution.

First 15 Minutes of an Incident

1. Stay Evidence-Driven



- Do not assume.
- Do not guess.
- Work with facts, logs, system state and evidence.
- Keep an open mind.
- Your goal is to understand what actually happened.

2. Confirm the Alert



- Check if the alert is real or a false positive.
- Validate using logs, monitoring tools or system checks.
- Understand what triggered the alert.
- Identify the scope and severity.

3. Record the Time



- Note the exact time you received the alert.
- Use UTC if possible.
- Record all key times during the investigation.
- A clear timeline helps you and the security team understand the sequence of events.

4. Identify Affected Assets



- Find out which systems, applications or services are affected.
- Identify the environment (production, staging, etc).
- Check if multiple systems are impacted.
- Understand who owns the affected assets.

5. Check What Changed



- Look for recent changes such as new files, services, users or configurations.
- Check logs, deployment history and system activity.
- Compare with normal behavior.
- Identify anything unusual around the time of the alert.

6. Preserve Useful Evidence



- Collect logs, system state and relevant artifacts.
- Capture volatile evidence before making changes.
- Document what you find.
- Avoid actions that could overwrite or destroy evidence.
- Follow your organization's evidence handling process.

7. Notify the Right People



- Inform your team, incident commander or on-call lead.
- Share a clear and concise summary of what you know.
- Escalate based on severity and impact.
- Include affected systems, observed behavior and initial findings.

8. Avoid Random Restarts and Destructive "Fixes"



- Do not restart systems without approval.
- Do not delete files or kill processes randomly.
- Do not make broad changes to configurations.
- These actions can destroy evidence and make the investigation harder.
- Always follow the incident response process.

First-Response Checklist

- ☑ Confirm the alert.
- ☑ Record the exact time.
- ☑ Identify affected assets.
- ☑ Check what changed.
- ☑ Preserve useful evidence.
- ☑ Notify the right people.
- ☑ Avoid random restarts or destructive fixes.
- ☑ Continue initial investigation.
- ☑ Document your findings.
- ☑ Follow escalation procedures.

Common Mistakes to Avoid



- Ignoring alerts.
- Assuming it is a false positive.
- Restarting systems too early.
- Deleting suspicious files.
- Making changes without approval.
- Not documenting what you did.

Key Takeaway



- The first 15 minutes are critical.
- Stay calm and follow a structured approach.
- Focus on evidence, not assumptions.
- Capture key information, preserve evidence and escalate when needed.
- Your actions at this stage can make the investigation easier and more effective.

Evidence Before Action

1. Why Evidence Matters



- Evidence helps you understand what happened, how it happened and who was involved.
- It allows you to confirm, not guess.
- It supports containment, eradication and recovery.
- It may be needed for legal, compliance or disciplinary action.
- Good evidence leads to better decisions and prevents repeat incidents.

2. Volatile vs Persistent Evidence

Volatile Evidence

- Stored in memory and can be lost when the system is powered off or rebooted.
- Examples:
  - RAM contents
  - Running processes
  - Network connections
  - Logged-in users
  - System state (uptime, running services)

Collect volatile evidence first because it can be lost easily.

Persistent Evidence

- Stored on disk and usually remains after a reboot.
- Examples:
  - Log files
  - Configuration files
  - User files
  - Application data
  - Disk contents
  - System binaries

Collect persistent evidence after volatile evidence and before making changes.

3. Key Types of Evidence

Memory (RAM)	• Running processes, loaded modules, malware in memory, decrypted data.
Processes	• What is running, command line, parent process, start time.
Connections	• Active network connections, listening ports, external IP addresses.
Logs	• Authentication logs, system logs, application logs, audit logs.
Files	• Suspicious files, modified files, configuration changes, timestamps.
Disk	• Full disk image, deleted files, malware artifacts, user data.

4. Why Rebooting Can Destroy Evidence



- Rebooting clears data from memory (RAM).
- Running processes and network connections are lost.
- Evidence such as malware, decrypted credentials or attacker tools may disappear.
- System timestamps and logs can change.
- It can make it harder to understand the full timeline and scope of the incident.

Only reboot when necessary and when approved by the incident response team.

5. Collection Before Modification



- Collect and preserve evidence before making any changes to the system.
- Changing files, stopping services or applying fixes can overwrite or destroy valuable evidence.
- Follow a structured collection process.
- Document what you collected, when you collected it and how you collected it.
- Use appropriate tools and methods approved by your organization.

6. Chain of Custody (Simple Introduction)



- Chain of custody is a record of who handled the evidence, when, where and why.
- It helps maintain the integrity and trustworthiness of the evidence.
- It shows that the evidence has not been tampered with.
- Even in non-legal situations, it is a good practice to document how evidence is collected, stored and transferred.

Key Takeaway



- Understand the different types of evidence.
- Collect volatile evidence first, then persistent evidence.
- Avoid unnecessary reboots.
- Always collect evidence before making changes.
- Keep a clear record of how evidence is handled.
- Evidence leads to better investigation, safer decisions and a stronger security posture.

Build an Incident Timeline

1. Why Timestamps Matter



- Timestamps show the sequence of events.
- They help you understand what happened, when it happened and how it spread.
- They allow you to correlate logs, system activity and user actions.
- Accurate timelines are essential for investigation, containment and reporting.

2. UTC and Timezone Awareness



- Use UTC (Coordinated Universal Time) when investigating.
- Different systems may use different timezones.
- Always check and convert timestamps to a single timezone (preferably UTC).
- Be aware of daylight saving time (DST) as it can cause confusion.

3. First Known Activity



- Identify the earliest sign of suspicious activity.
- This could be a failed login, a new user, an unusual process or a strange network connection.
- Start your timeline from the first known event, not when you detected it.

4. Detection Time



- When was the incident detected or alerted?
- What tool generated the alert (e.g. SIEM, IDS, monitoring tool)?
- Record the exact time the alert was received.
- Compare this with the first known activity to understand how long the incident was active before detection.

5. Authentication Activity



- Look for successful and failed logins.
- Check for logins from unusual IP addresses or locations.
- Identify new or unknown user accounts.
- Look for privilege escalation (e.g. sudo or admin access).

6. Process Execution



- Identify suspicious processes that were started.
- Check the command line, parent process and start time.
- Look for unknown binaries or scripts.
- Correlate process execution with other events in the timeline.

7. File Changes



- Look for new, modified or deleted files.
- Check important directories such as /etc, /var, user home directories and web roots.
- Note the exact timestamp of the changes.
- Identify suspicious files such as malware, scripts or modified configuration files.

8. Network Activity



- Check for unusual outbound connections.
- Identify connections to unknown IP addresses or domains.
- Look at the destination IP, port, protocol and time.
- Detect data exfiltration or command and control traffic.
- Correlate network activity with process and file events.

9. Containment



- Record the exact time containment actions were taken.
- Examples: isolating the host, blocking an IP, disabling an account or stopping a service.
- Include who took the action and what was done.
- This helps show when the threat was contained.

10. Simple Timeline Example

Time (UTC)	Event	Details
2026-10-14 12:01:15	First known activity	Failed SSH login from 203.0.113.45
2026-10-14 12:05:22	Authentication success	User admin logged in from same IP
2026-10-14 12:08:10	Process execution	Suspicious process /usr/bin/.update started
2026-10-14 12:10:44	File change	New file /tmp/backdoor.sh created
2026-10-14 12:12:30	Network activity	Outbound connection to 198.51.100.23:443
2026-10-14 12:18:10	Detection time	SIEM alert triggered
2026-10-14 12:25:00	Containment	Host isolated from network

Key Takeaway



- A timeline helps you see the full picture, not just a single event.
- Use UTC and be consistent with timezones.
- Correlate logs, authentication, processes, files and network activity.
- Include both attacker activity and your response actions.
- A clear timeline makes investigation, communication and reporting much easier.

Linux Host Triage

1. Start with System State



- Get an overview of the current state before going deeper.
- Check who is logged in, what is running, network connections and system load.
- This helps you understand what is normal and identify anything unusual.
- Observe and collect information before making any changes to the host.

2. who

```
$ who
user1    pts/0        2026-10-14 10:21
user2    pts/1        2026-10-14 11:03
```

- Shows currently logged-in users.
- Displays the terminal, source and login time.
- Useful for spotting unexpected users or sessions.

3. w

```
$ w
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU WHAT
user1    pts/0    192.168.1.10     10:21    1.00s  0.02s  0.01s sshd
user2    pts/1    10.0.0.5         11:03    2.00s  0.10s  0.05s bash
```

- Shows who is logged in and what they are doing.
- Displays the source IP, login time, idle time and current command.
- Helps you identify suspicious activity, multiple sessions or unexpected users.

4. ps aux

```
$ ps aux | head -5
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.1  17200  9868 ?        Ss   10:15   0:02 /sbin/init
root       423  0.0  0.2  54320 12032 ?        Ss   10:15   0:00 /usr/sbin/sshd
user       845  0.1  0.3  98764 23456 ?        Ssl  10:20   0:01 /usr/bin/python3 app.py
user      1021  0.0  0.1  43920 10240 pts/0    Ss   10:21   0:00 -bash
```

- Shows all running processes.
- Displays the user, PID, CPU and memory usage, start time and command.
- Helps you identify unusual or suspicious processes.
- Look for processes running from unexpected locations or with strange names.

5. ss -tulpn

```
$ ss -tulpn
Netid  State   Recv-Q  Send-Q  Local Address:Port   Peer Address:Port   Process
tcp    LISTEN  0       128     0.0.0.0:22           0.0.0.0:*           users:("sshd",pid=423)
tcp    LISTEN  0       128     0.0.0.0:80           0.0.0.0:*           users:("nginx",pid=612)
tcp    ESTAB   0       0       192.168.1.10:54321   93.184.216.34:443   users:("curl",pid=984)
udp    UNCONN  0       0       0.0.0.0:68           0.0.0.0:*           users:("dhclient",pid=512)
```

- Shows network connections and listening ports.
- Displays the local and remote addresses, port numbers and the process using the connection.
- Helps you identify unexpected listening services or suspicious outbound connections.

6. uptime

```
$ uptime
11:32:15 up 2 days,  4:15,  2 users,  load average: 0.12, 0.08, 0.05
```

- Shows how long the system has been running.
- Displays the number of logged-in users.
- Shows the system load average.
- Helps you understand recent reboots and overall system activity.

7. last

```
$ last -n 5
user1    pts/0        192.168.1.10     Wed Oct 14 10:21   still logged in
user2    pts/1        10.0.0.5         Wed Oct 14 11:03   still logged in
admin    pts/0        203.0.113.25     Wed Oct 14 09:12   - 10:18 (01:06)
user3    pts/2        192.168.1.20     Tue Oct 13 18:44   - 22:11 (03:27)
```

- Shows recent user login history.
- Displays the username, terminal, source IP and login time.
- Helps you identify unusual or unauthorized logins.
- Use lastb to view failed login attempts.

8. What Each Command Can Reveal

Command	What It Can Reveal
who	Currently logged-in users and their terminals.
w	Logged-in users, source IP and what they are doing.
ps aux	All running processes and their details.
ss -tulpn	Network connections, listening ports and the process using them.
uptime	System uptime, number of users and system load.
last	Recent login history, useful for spotting suspicious access.

9. Observe Before Changing the Host



- Collect as much information as possible before making changes.
- Avoid unnecessary restarts or killing processes.
- Document what you find, including timestamps and command output.
- Changing the system too early can destroy evidence and make investigation harder.
- If you need to take action, follow your organization's incident response process.

Key Takeaway

- Start with ahigh-level view of the system state.
- Use these commands to quickly identify suspicious activity.
- Observe and gather evidence before making any changes to the host.

Investigating Linux Logs

1. What Logs Can Tell You



- Logs record what happened on the system.
- They show user activity, application behavior, errors, security events and system changes.
- Logs help you understand the timeline, find the cause and identify suspicious activity.
- Always check multiple log sources and correlate the information.

2. Authentication Logs



- Authentication logs show login attempts, successful logins, failed logins, and privilege use.
- Common files:

```
/var/log/auth.log   (Ubuntu/Debian)
/var/log/secure     (RHEL/CentOS)
```

```
# View failed login attempts
$ sudo grep "Failed password" \
  /var/log/auth.log

# View successful logins
$ sudo grep "Accepted password" \
  /var/log/auth.log
```

3. journalctl



- journalctl reads logs from systemd services and the system journal.
- It includes kernel messages, service logs and system events.
- Useful options:

```
# View recent logs
$ sudo journalctl -xe

# View logs for a specific service
$ sudo journalctl -u ssh -n 50

# Filter by time
$ sudo journalctl --since "2026-10-14
  10:00" --until "2026-10-14 12:00"
```

4. dmesg



- dmesg shows kernel messages, hardware events and low-level system activity.
- It can reveal driver issues, hardware errors and signs of tampering.
- Useful for early boot information and system changes.

```
# View recent kernel messages
$ dmesg | tail -n 50

# Search for errors
$ dmesg | grep -i error

# Follow kernel messages in real time
$ dmesg -w
```

5. Application and Service Logs



- Applications and services write their own logs.
- These logs can show errors, unusual activity, successful operations and attacks targeting the application.
- Common locations:

```
/var/log/nginx/
/var/log/apache2/
/var/log/mysql/
/var/log/
/var/log/syslog
```

```
# View Nginx access log
$ sudo tail -n 50 /var/log/nginx/access.log

# Search for suspicious IP
$ sudo grep "192.168.1.100" /var/log/nginx/access.log
```

6. Searching with grep



- grep helps you find specific text in log files.
- Use it to search for IP addresses, usernames, errors or other keywords.
- It is case-sensitive by default. Use -i for case-insensitive search.

```
# Search for a specific IP
$ grep "192.168.1.100" /var/log/auth.log

# Search for a username
$ grep "user1" /var/log/auth.log

# Case-insensitive search
$ grep -i "error" /var/log/syslog

# Show line numbers
$ grep -n "Failed" /var/log/auth.log
```

7. Correlating Timestamps



- Do not read logs randomly.
- Look at events across different logs using the same time range.
- Compare authentication logs, system logs, application logs and network logs.
- This helps you see the full picture and understand the sequence of events.
- Use UTC or convert timezones to keep timestamps consistent.

8. Example: Correlating Log Events

Time (UTC)	Log Source	Event
10:15:01	auth.log	Failed SSH login from 192.168.1.100
10:16:22	auth.log	Successful SSH login for user1
10:17:05	journalctl	sudo command executed
10:18:30	nginx access.log	Suspicious request to /admin
10:19:12	dmesg	New network interface detected

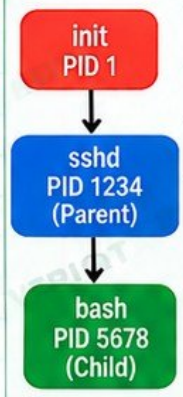
Key Takeaway



- Logs tell a story when you look at them together.
- Use the right log source for the information you need.
- Search with grep and focus on relevant time ranges.
- Always correlate timestamps instead of reading logs randomly.
- Logs are one of the most important tools in incident investigation.

Suspicious Processes

1. Process, PID, Parent Process and Command Line



- A process is a running program.
- Each process has a unique PID (Process ID).
- Every process has a parent process (PPID) that started it.
- The command line shows exactly how the process was started, including arguments.
- This information helps you understand what the process is, where it came from and whether it is expected.

2. Expected vs Unexpected Processes

Common Expected Processes

- systemd (PID 1)
- sshd (SSH service)
- nginx, apache2) (web servers)
- mysqld (database)
- docker, containerd
- kubelet (Kubernetes)
- cron (scheduled tasks)
- rsyslog, journald (llogging)

Examples of Suspicious or Unexpected Processes

- Unknown or strange process names.
- Processes running from unusual locations (e.g. /tmp, /dev/shm).
- Processes with random or obfuscated names.
- Unexpected network tools (e.g, nc, ncat, socat).
- Processes running as root without a clear reason.

3. ps (Process Status)

```
$ ps aux | head -5
USER  PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root    1  0.0  0.1  17000  9688 ?        Ss   10:01   0:02 /sbin/init
root  423  0.0  0.2  54320 12032 ?        Ss   10:02   0:00 /usr/sbin/sshd
www   845  0.1  0.3  98764 23456 ?        Ssl  10:05   0:01 /usr/sbin/nginx
user 1021  0.0  0.1  43920 10240 pts/0    S+   10:10   0:00 -bash
```

- Shows all running processes.
- Displays the user, PID, CPU and memory usage, parent information is not shown by default, and the command line.
- Use options like ps aux, ps -ef or ps -eo pid,ppid,cmd for more details.

4. pstree (Process Tree)

```
$ pstree -p | head -10
systemd(1)
    ├─sshd(423)
    │    └─bash(1021)
    │         └─ps(2045)
    ├─nginx(845)
    ├─mysqld(912)
    ├─cron(1003)
    └─rsyslogd(678)
```

- Shows the parent-child relationship between processes.
- Helps you see how a suspicious process was started.
- Useful for identifying unusual process trees or processes started by unexpected parents.

5. top (Real-Time Process View)

```
$ top -b -n 1 | head -10
PID  USER     %CPU  %MEM   COMMAND
1021 user     25.3   1.2   python
845  www      12.1   0.8   nginx
912  mysql     5.0   2.1   mysqld
423  root      1.2   0.5   schd
1003 root      0.3   0.1   cron
```

- Shows running processes in real time.
- Displays CPU and memory usage.
- Helps you quickly spot processes using high resources.
- Useful during an incident to identify sudden or unusual activity.

6. /proc (Process Information)

```
$ ls /proc/1021
attr       fd        maps        net        status
autogroup  fdinfo    mem         ns         task
cmdline    gid_map   mountinfo   root       uid_map
cwd        io        mounts      sched      environ
exe        limits    mountstats  stat       ...
```

- /proc/<PID>/ contains detailed information about a process.
- Important files:
  - cmdline → Shows the full command line.
  - status → Shows user, memory, state, etc.
  - environ → Shows environment variables.
  - fd → Shows open files and connections.
  - maps → Shows loaded libraries and memory.

7. Why High CPU Alone Does Not Prove Compromise



- A high CPU process can be normal, for example a backup, update, compilation or heavy application.
- Attackers can also use high CPU processes, but high CPU alone is not evidence of compromise.
- You must look at the process name, command line, parent process, user, network connections and what the process is doing.
- Always investigate and gather evidence before taking action.

8. Investigate Before Killing a Process



- Do not kill a process immediately unless there is a clear and urgent reason.
- Check the command line, parent process, open files, network connections and what the process is doing.
- Killing the wrong process can cause service outages or data loss.
- Collect the necessary information and document your findings.
- If it is confirmed to be malicious and you have approval, then take action.

Key Takeaway



- Understand process details: name, PID, parent process and command line.
- Use ps, pstree, top and /proc to gather information.
- High CPU alone does not prove compromise.
- Always investigate and collect evidence before killing a process.
- This helps you make the right decision and avoids unnecessary disruptions.

Network Connections During an Incident

1. Local and Remote Addresses



- Every network connection has a local address and a remote address.
- The local address is your server's IP and port.
- The remote address is the other system's IP and port.
- These help you understand where the connection is coming from and where it is going.

Field	Meaning	Example
Local Address	Your server's IP and port	192.168.1.10:22
Remote Address	Other system's IP and port	203.0.113.45:54321

2. Listening Ports



- Listening ports are waiting for incoming connections.
- They are used by services such as SSH, HTTP, database servers and application services.
- During an incident, check if there are unexpected services listening on new or unusual ports.

```
$ ss -tulpn
Netid  State    Local Address:Port    Process
tcp    LISTEN     0.0.0.0:22         sshd
tcp    LISTEN     0.0.0.0:80         nginx
tcp    LISTEN     127.0.0.1:3306     mysqld
```

3. Established Connections



- Established connections show active communication between your server and other systems.
- They can be normal or suspicious depending on the remote address, port and process.
- Check established connections to identify unexpected or unknown destinations.

```
$ ss -tanp | head -5
State   Local Address:Port   Remote Address:Port   Process
ESTAB   192.168.1.10:22      203.0.113.45:54321    sshd
ESTAB   192.168.1.10:443     198.51.100.25:443     nginx
ESTAB   10.0.0.5:3306        10.0.0.20:54321       mysqld
```

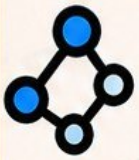
4. ss (Socket Statistics)



- ss shows detailed information about network connections.
- It can show listening ports, established connections and the process using the connection.
- It is newer and often preferred over netstat.

```
# Show all connections with process information
$ ss -tulpn
# Show established connections
$ ss -tanp
# Show listening ports
$ ss -ltnp
```

5. ip (Network Information)



- ip shows network interfaces, IP addresses and routing tables.
- It helps you understand the network configuration during an incident.
- Check for unexpected interfaces, IP addresses or routes.

```
# Show network interfaces
$ ip addr show
# Show routing table
$ ip route show
# Show detailed information about an interface
$ ip -d link show eth0
```

6. lsof -i (Processes Using Network)



- lsof -i shows which processes are using network connections.
- It helps you map a process to a specific IP address and port.
- This is useful for identifying which process is making suspicious connections.

```
# Show all processes using network connections
$ lsof -i
# Show processes listening on ports
$ lsof -i -P -n | grep LISTEN
# Show process details for a specific port
$ lsof -i :443
```

7. Unexpected Outbound Connections



- Check for outbound connections to unknown IP addresses or unusual countries.
- Look for connections to high-risk ports or suspicious domains.
- These can indicate malware, data exfiltration or a compromised host.

```
# Find established outbound connections
$ ss -tanp | grep ESTAB
# Show only remote addresses
$ ss -tan | awk '{print $5}' | sort | uniq -c
```

8. Connection Refused vs Suspicious Connection

Connection Refused (Normal)

- Means the target host is reachable but there is no service listening on that port.
- Common during normal scanning, misconfiguration or when a service is down.
- Usually not an indicator of compromise on its own.

```
$ nc -zv 192.0.2.10 22
Connection refused
```

Suspicious Connection

- Connections to unknown or unexpected IP addresses.
- Connections to unusual ports or known malicious infrastructure.
- Regular outbound connections to rare destinations.
- Connections associated with suspicious processes.

```
$ nc -zv 198.51.100.25 4444
succeeded!
```

9. Process ↔ Port ↔ Connection Mental Model

Process
(e.g. nginx, python, unknown)

↓

Port
(e.g. 80, 443, 3306)

↓

Connection
(Local ↔ Remote)
(e.g. 192.168.1.10:443 -- 198.51.100.25:443)

- A process uses a port to create a connection.
- By checking the process, port and remote address together, you can understand what is happening and identify suspicious activity.

Key Takeaway



- Check local and remote addresses, listening ports and established connections.
- Use ss, ip and lsof -i to gather network information.
- Look for unexpected outbound connections.
- Understand the difference between connection refused and suspicious connections.
- Always map the process, port and connection together before taking action.

Authentication Investigation

1. Successful vs Failed Authentication



- Successful authentication means a user has logged in.
- Failed authentication means the login attempt was rejected.
- Look for patterns of repeated failures, unusual times, or logins from unexpected locations.
- Both successful and failed logins are important for investigation.

2. SSH Activity



- SSH is commonly targeted by attackers.
- Check for successful and failed SSH logins.
- Look for logins from unknown IP addresses, unusual times or unknown users.

```
# Check successful SSH logins
$ grep "Accepted" /var/log/auth.log
# Check failed SSH logins
$ grep "Failed password" /var/log/auth.log
# Show SSH activity with timestamps
$ sudo journalctl -u ssh -n 50
```

3. Unexpected Users



- Look for user accounts that you do not expect.
- Check for recently created users.
- Identify users with unusual names or UIDs.
- Verify if the users have legitimate business reasons to be on the system.

```
# List all users
$ cut -d: -f1 /etc/passwd
# Check recently created users
$ sudo lastlog | grep -v "Never" | head
# Show user details
$ id username
```

4. Privilege Escalation



- Privilege escalation happens when a user gains higher permissions than intended.
- Look for use of sudo, su or changes to admin groups.
- Check for unusual sudo usage or commands run as root.
- Attackers often escalate privileges after gaining initial access.

```
# Check sudo group members
$ getent group sudo
# Check recent privilege escalation
$ grep "sudo" /var/log/auth.log
# Look for su to root
$ grep "su:" /var/log/auth.log
```

5. last (Successful Logins)



- Shows a list of successful logins.
- Displays the username, terminal, source IP and time.
- Helps identify when a user logged in and from where.

```
$ last -n 10
user1   pts/0   192.168.1.10   Tue Oct 14 10:21
user2   pts/1   10.0.0.5       Tue Oct 14 09:18
admin   pts/0   203.0.113.25   Mon Oct 13 18:44
user3   tty1    192.168.1.20   Mon Oct 13 17:02
```

6. lastb (Failed Logins)



- Shows a list of failed login attempts.
- Useful for detecting brute force attacks.
- Look for repeated failures from the same IP address or multiple IP addresses.

```
$ sudo lastb -n 10
user1   ssh:notty   192.168.1.100   Tue Oct 14 10:05
user2   ssh:notty   203.0.113.45    Tue Oct 14 09:42
invalid ssh:notty   198.51.100.23   Tue Oct 14 09:40
user3   ssh:notty   10.0.0.15       Mon Oct 13 22:11
```

7. who (Currently Logged In Users)



- Shows who is currently logged in to the system.
- Displays the username, terminal, login time and source IP (if available).
- Helps you spot unexpected or suspicious active sessions.

```
$ who
user1   pts/0   2026-10-14 10:21   (192.168.1.10)
user2   pts/1   2026-10-14 11:03   (10.0.0.5)
admin   tty1    2026-10-14 09:18   (:0)
```

8. sudo Activity



- Check what commands were run with sudo.
- Identify when, who and which commands were executed.
- Look for unusual or unexpected commands.

```
# Check sudo activity in auth log
$ grep "sudo:" /var/log/auth.log
# Use journalctl for detailed view
$ sudo journalctl _COMM=sudo -n 50
# Example output
Oct 14 10:21:33 server sudo: user1 :
TTY=pts/0 ; PWD=/home/user1 ; USER=root ;
COMMAND=/bin/systemctl restart nginx
```

9. Repeated Failures Followed by a Successful Login



- Attackers often try many failed logins before succeeding.
- Look for multiple failed attempts from the same IP, followed by a successful login.
- This pattern is a strong indicator of a brute force attack.

```
# Find repeated failures then success
$ grep -E "Failed password|Accepted" \
  /var/log/auth.log | grep "192.168.1.100"
```

Key Takeaway



- Investigate both successful and failed authentication events.
- Check SSH activity, user accounts, privilege escalation and sudo usage.
- Look for patterns such as repeated failures followed by a successful login.
- Use last, lastb and who to understand who accessed the system and when.
- Authentication logs provide critical evidence during an incident investigation.

Filesystem and File Evidence

1. Unexpected Files



- Look for files that you do not expect on the system.
- Check unusual file names, locations or file types.
- Be alert for hidden files (e.g. starting with .).
- Look for scripts, binaries or web shells that attackers may have uploaded.
- Compare with a known good baseline if available.

```
# Find recently created files (last 1 day)
$ find / -type f -mtime -1 2>/dev/null
# Look for hidden files
$ find / -type f -name ".*" 2>/dev/null
# Find suspicious file types
$ find / -type f \( -name *.sh" -o -name *.py" \)
  2>/dev/null
```

2. Modified Configuration



- Attackers often modify configuration files to maintain access or change system behavior.
- Check important configuration files for unexpected changes.
- Compare modification times with the time of the incident.
- Look for added users, changed settings or malicious entries.

```
# Check recently modified config files
$ find /etc -type f -mtime -7 2>/dev/null
# View a config file with line numbers
$ nl -ba /etc/ssh/sshd_config | less
```

3. Permissions and Ownership



- Check file permissions to identify unusual or overly permissive files.
- Look for files owned by unexpected users or groups.
- Attackers may change permissions to maintain access or hide files.
- Focus on sensitive directories like /etc, /usr, /var and web directories.

```
# List permissions and ownership
$ ls -la /etc
# Find world-writable files
$ find / -type f -perm -0002 2>/dev/null
# Find files not owned by root in /etc
$ find /etc -type f ! -user root 2>/dev/null
```

4. Modification Times



- File timestamps show when a file was created, modified or accessed.
- Compare modification times with the incident timeline.
- Look for files changed around the time of suspicious activity.
- Attackers may modify existing files instead of creating new ones.

```
# List files with detailed timestamps
$ ls -la --time-style=full-iso /etc
# Find files modified in the last 24 hours
$ find / -type f -mtime -1 2>/dev/null
# Show last modification time using stat
$ stat /etc/passwd
```

5. stat (File Information)



- The stat command shows detailed information about a file.
- It displays size, permissions, ownership and timestamps.
- Useful for investigating specific files during an incident.

```
# Get detailed file information
$ stat /etc/passwd
  File: /etc/passwd
  Size: 1651        Blocks: 8      IO Block: 4096
Device: 802h/2050d Inode: 262144   Links: 1
Access: 2026-10-14 09:12:01.123 +0000
Modify: 2026-10-14 08:55:10.456 +0000
Change: 2026-10-14 08:55:10.456 +0000
 Birth: -
```

6. find (Search for Files)



- The find command helps you search for files based on different criteria.
- You can search by name, type, size, owner, permission or modification time.
- Useful for locating suspicious files during an investigation.

```
# Find files by name
$ find / -type f -name "suspicious.sh"
# Find large files (over 100MB)
$ find / -type f -size +100M 2>/dev/null
# Find files modified in last 7 days
$ find / -type f -mtime -7 2>/dev/null
# Find files owned by a specific user
$ find / -type f -user www-data 2>/dev/null
```

7. ls -la (List Files and Permissions)



- The ls -la command shows a detailed list of files and directories.
- It displays permissions, ownership, size and modification time.
- Useful for quickly checking for unusual files or permission changes.

```
$ ls -la /var/www/html
total 24
drwxr-xr-x  3 root     root     4096 Oct 14 10:20 .
drwxr-xr-x 12 root     root     4096 Oct 10 08:10 ..
-rw-r--r--  1 root     root     1234 Oct 14 09:55 index.php
-rwxr-xr-x  1 www-data www-data 5320 Oct 14 09:58 shell.sh
```

8. Hashing Evidence with sha256sum



- The sha256sum command creates a unique hash of a file.
- The hash acts like a fingerprint for the file.
- If the file is changed, the hash will be different.
- Useful for verifying file integrity and preserving evidence.

```
# Generate a hash for a file
$ sha256sum /path/to/suspicious.sh
3f2a4e8c9d2b7c5f1e6a9d8c0b1e2f3a4
 5b6c7d8e9f0a1b2c3d4e5f6a7b8c9d0  suspicious.sh
```

9. Why Investigators Hash Collected Files



- Hashing ensures the file has not been changed since it was collected.
- It helps maintain the integrity of evidence.
- Investigators can compare hashes later to confirm the file is the same.
- Important for legal and forensic investigations.

Key Takeaway



- Look for unexpected files, modified configurations and unusual permissions.
- Use ls -la, stat and find to gather file information.
- Check modification times and compare with the incident timeline.
- Use sha256sum to create hashes and preserve evidence.
- File evidence helps you understand what happened and who may have made changes.

Indicators of Compromise

1. What an IOC Is



- An Indicator of Compromise (IOC) is a piece of evidence that may suggest malicious activity.
- IOCs can be found in logs, network traffic, files, user accounts, processes and system configuration.
- An IOC does not always mean a confirmed breach, but it should be investigated.

Examples of IOCs

- Suspicious IP address
- Malicious domain
- Known file hash
- Suspicious filename
- Unexpected user account
- Unusual process
- Indicators in logs

2. Suspicious IP Addresses



- Suspicious IP addresses may be known malicious IPs, come from unusual locations, or show unusual behavior.
- Check incoming and outgoing connections for suspicious IPs.
- Use threat intelligence sources to verify.

```
# Find connections to a specific IP
$ ss -tupn | grep 203.0.113.45
# Search logs for an IP address
$ grep "203.0.113.45" /var/log/auth.log
# Check hostname and reputation (example)
$ whois 203.0.113.45
```

3. Domains



- Malicious or suspicious domains may be used for command and control (C2), malware download or data exfiltration.
- Look for unusual domain lookups or connections to unknown domains.
- Compare domains against threat intelligence lists.

```
# Search logs for a domain
$ grep "malicious-domain.com" /var/log/syslog
# Check DNS queries (if logged)
$ grep "malicious-domain.com" /var/log/dns.log
# Resolve a domain to check IP
$ dig malicious-domain.com
```

4. File Hashes



- A file hash is a unique fingerprint of a file.
- Known malicious hashes can indicate compromised files (malware, web shells, backdoors).
- Compare file hashes with trusted sources or threat intelligence feeds.

```
# Generate a hash for a file
$ sha256sum suspicious.sh
# Compare with a known malicious hash
$ grep "d41d8cd98f00b204e9800998ecf8427e"
  hashes.txt
```

5. Filenames



- Attackers may use suspicious filenames (e.g. disguised binaries, web shells, or files with unusual extensions).
- Look for unexpected files in common directories such as /tmp, /var/tmp, /opt or web directories.

```
# Find suspicious files by name
$ find / -type f -name "*.sh" 2>/dev/null
# Look for common web shells
$ find /var/www -type f -name "*shell*"
  2>/dev/null
```

6. User Accounts



- Unexpected or unknown user accounts may be created by attackers.
- Look for accounts with unusual names, UIDs or privileges.
- Check for recently created accounts and review their activity.

```
# List all users
$ cut -d: -f1 /etc/passwd
# Find recently created accounts
$ sudo lastlog | grep -v "Never" | head
# Check for accounts with UID 0 (root)
$ awk -F: '($3 == 0) {print $1}' /etc/passwd
```

7. Processes



- Suspicious processes may indicate malware, reverse shells or unauthorized tools.
- Look for unusual process names, processes running from unexpected locations, or processes with strange command lines.

```
# List running processes with full command line
$ ps auxf
# Search for suspicious process names
$ ps aux | grep -i "nc\|ncat\|socat\|python"
# Check process location
$ readlink -f /proc/<PID>/exe
```

8. Why One IOC Is Not Proof



- A single IOC can be a false positive (e.g. a legitimate tool, internal scanner, or good traffic from a shared IP).
- Context matters. Always investigate and correlate with other evidence.
- Look for multiple related IOCs and patterns before drawing a conclusion.

9. IOC → Search → Correlate → Validate



- Identify a suspicious IP, domain, file hash, filename, user or process.
- Search logs, systems and tools for the IOC.
- Look for related activity across multiple sources (e.g. logs, network, files, processes).
- Determine if the activity is malicious, benign or inconclusive.

Key Takeaway



- IOCs help you detect and investigate suspicious activity.
- Check IP addresses, domains, file hashes, filenames, user accounts and processes.
- One IOC is not proof of compromise. Always investigate and correlate evidence.
- Follow the IOC → Search → Correlate → Validate approach.

Cloud Incident Investigation

1. Why Cloud Investigation Differs from One Linux Server



- Cloud environments have many interconnected services, shared infrastructure and multiple access methods.
- Changes can happen quickly and at scale.
- Logs, identity data and API activity are critical evidence sources.
- You must investigate the control plane (management layer), not just individual servers.

In the cloud, the full picture comes from identity, API activity, logs and resource changes, not just one server.

2. Identity Activity



- Check for suspicious sign-ins, unusual locations or new devices.
- Look for use of compromised credentials or unexpected IAM users.
- Review changes to users, roles and permissions.
- Check for privilege escalation or new access keys.

```
# AWS: View recent sign-in activity
aws cloudtrail lookup-events \
  --lookup-attributes \
  AttributeKey=Username,AttributeValue=user1

# Azure: Sign-in logs (example with CLI)
az account get-access-token
# then query sign-in logs in Azure Portal
```

3. API Activity



- Attackers often use cloud APIs to enumerate resources, change settings or create new resources.
- Look for unusual or high volume API calls.
- Check for API calls from unexpected IP addresses or tools.
- Review failed and successful API activity.

```
# AWS: Look at recent API activity
aws cloudtrail lookup-events \
  --max-results 10 \
  --query 'Events[].{Time:EventTime,
   Name:EventName, User:Username}'
# Filter for specific API calls
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,
 AttributeValue=CreateUser
```

4. Audit Logs



- Cloud audit logs record who did what, when and from where.
- They show management events and resource changes.
- Enable and review audit logs regularly.
- Keep logs centralized and retained for investigations.

```
# AWS CloudTrail (view recent events)
aws cloudtrail lookup-events --max-results 10
# Azure Activity Log (CLI example)
az monitor activity-log list --max-events 10
# GCP Audit Logs (gcloud example)
gcloud logging read "logName=\"cloudaudit\"" \
 --limit 10
```

5. Security Groups and Firewall Changes



- Check for changes to security groups, network ACLs and firewall rules.
- Look for rules that allow unexpected access (e.g. 0.0.0.0/0).
- Identify who made the changes and when.
- Compare current rules with previous known good state.

```
# AWS: List security group rules
aws ec2 describe-security-groups \
  --group-ids sg-1234567890
  --query 'SecurityGroups[].IpPermissions'
# Azure: Show network security group rules
az network nsg rule list --nsg-name MyNSG
```

6. Storage Access



- Check for unusual access to object storage (S3, Blob Storage, GCS).
- Look for downloads, deletions or public access changes.
- Review access logs to identify who accessed which files and when.
- Check for newly created or modified buckets.

```
# AWS: View S3 access logs (example in CloudTrail)
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,
   AttributeValue=GetObject
# List S3 buckets
aws s3 ls
```

7. Unexpected Resource Creation



- Look for resources that were created without authorisation.
- Check for unknown VMs, containers, databases, functions or storage.
- Identify who created them, from where and why.
- Review tags, naming patterns and creation time.

```
# AWS: Find recently created EC2 instances
aws ec2 describe-instances --query \
  'Reservations[].Instances[].{ID:InstanceId,
   State:State.Name, LaunchTime:LaunchTime}'
```

8. Cloud Control Plane as an Evidence Source



- The cloud control plane records all management actions and configuration changes.
- It provides authoritative evidence of who did what, when and from where.
- Use audit logs, activity logs and configuration history to reconstruct the incident timeline.
- Do not rely only on data from individual servers.

9. Key Takeaway



- Cloud investigation requires a broader view than a single server.
- Focus on identity activity, API calls, audit logs, network changes, storage access and new resources.
- The cloud control plane is a critical evidence source.
- Always correlate multiple sources to understand the full picture.

Container and Kubernetes Incidents

1. Container vs Host Evidence



Container Evidence

- Container filesystem
- Container logs
- Running processes
- Environment variables
- Network connections
- Mounted volumes



Host Evidence

- Host logs
- Container runtime logs
- Node processes
- Network activity
- File system changes
- Kubernetes components
- Other containers on node

Always investigate both container and host. An incident may affect one container, multiple containers, or the underlying host.

2. Pod Behavior



- Check pod status (Running, Pending, CrashLoopBackOff, Error, Completed).
- Look for unusual behavior or unexpected state changes.
- Verify the number of containers in the pod.
- Check if the pod is behaving differently from normal.

```
# Check pod status in a namespace
kubectl get pods -n <namespace>
# Detailed information about a pod
kubectl describe pod <pod-name> -n <namespace>
# Check container status inside the pod
kubectl get pod <pod-name> -n <namespace>
-o wide
```

3. Restarts



- Frequent restarts may indicate a crash, misconfiguration or a malicious activity.
- Check the restart count and look for patterns.
- Identify what is causing the restarts by checking events and logs.
- Repeated restarts can hide attacker activity.

```
# Show restart count for all pods
kubectl get pods -n <namespace>
# Show detailed information including restarts
kubectl describe pod <pod-name> -n <namespace>
# Watch pod events in real time
kubectl get events -n <namespace> –sort-by=.lastTimestamp
```

4. Kubernetes Events



- Kubernetes events show what happened to the pod, such as scheduling, image pulls, crashes and restarts.
- Look for warnings, errors or unusual events.
- Events help you understand the timeline of what happened.

```
# View recent events in a namespace
kubectl get events -n <namespace> --sort-by=.lastTimestamp
# Filter events for a specific pod
kubectl get events -n <namespace> --field-selector
involvedObject.name=<pod-name>
# Describe pod to see related events
kubectl describe pod <pod-name> -n <namespace>
```

5. Useful kubectl Commands



- kubectl get pods – list pods.
- kubectl describe pod – detailed information.
- kubectl logs – view container logs.
- Use these commands together to understand what is happening.

```
# List all pods
kubectl get pods -n <namespace>
# Describe a specific pod
kubectl describe pod <pod-name> -n <namespace>
# View logs from a container
kubectl logs <pod-name> -n <namespace>
# Investigate the real time
kubectl logs -f <pod-name> -n <namespace>
```

6. Why Not Delete a Suspicious Pod



- Deleting a pod immediately can destroy important evidence such as logs, file system changes and runtime state.
- You may lose information about what the attacker did and how they gained access.
- Investigate and collect evidence before taking disruptive actions.

Always capture logs, describe output, events and relevant data before deleting or restarting a suspicious pod.

7. Workload → Pod → Node → Cluster Path



Workload (Deployment, StatefulSet, DaemonSet, Job, CronJob)

Pod (One or more containers)

Node (Worker node running the pod)

Cluster (Entire Kubernetes environment)

- Start from the workload to understand the desired state.
- Investigate the pod for actual behavior and logs.
- Check the node for host-level evidence and other affected pods.
- Look at the cluster for broader impact and related activity.

8. Example Investigation Flow

1. Check workload (deployment/statefulset)
2. Check pod status and restarts
3. Describe the pod and review events
4. Check container logs
5. Investigate the node if needed
6. Look for related activity across the cluster

9. Key Takeaway



- Investigate both container and host evidence.
- Check pod behavior, restarts and events.
- Use kubectl get pods, kubectl describe pod and kubectl logs.
- Do not delete a suspicious pod immediately.
- Follow the Workload → Pod → Node → Cluster path.
- Collect evidence before taking action to understand the full scope of the incident.

Containment Without Making It Worse

1. What Containment Means



- Containment means limiting the attacker's ability to cause further damage.
- It reduces risk, buys time for investigation and recovery, and helps keep the incident under control.
- Containment is not complete remediation, but a critical early step.

The goal is to stop or limit the spread while preserving evidence and maintaining essential services.

2. Isolate vs Terminate



- Keep the system running but cut off network access.
- Prevent further damage while preserving evidence.
- Useful when you need logs, memory or data for investigation.



- Stop the system completely.
- Use when the system poses immediate and severe risk.
- May result in loss of volatile evidence (e.g. memory, running processes).

Choose the option that reduces risk without destroying critical evidence.

3. Disable Compromised Credentials



- Immediately disable compromised user accounts, API keys, access tokens and service accounts.
- Rotate passwords and keys.
- Revoke active sessions and tokens.
- Check for other accounts that may have been affected.
- Look for signs of privilege escalation using the compromised credentials.

```
# Example actions (Linux/cloud)
$ passwd -l username
# Revoke cloud access key
aws iam update-access-key --status Inactive
```

4. Restrict Network Access



- Block malicious IP addresses.
- Restrict inbound and outbound traffic.
- Update firewall rules, security groups or network ACLs.
- Limit access to only necessary systems and users.
- Monitor for continued suspicious traffic.

```
# Example: block an IP with iptables
$ sudo iptables -A INPUT -s 203.0.113.45 -j DROP
# Example: update a security group (AWS CLI)
aws ec2 revoke-security-group-ingress \
--group-id sg-12345678 --protocol tcp --port 22 \
 --cidr 203.0.113.0/24
```

5. Preserve Evidence



- Collect and preserve logs, memory, disk images and relevant files.
- Avoid actions that overwrite or delete evidence.
- Document everything you do, including time, commands and reasons.
- Preserve evidence from both the affected system and related systems (e.g. cloud logs, network devices).

Containment should be done in a way that maintains the integrity of evidence for further investigation.

6. Temporary vs Long-Term Containment



Temporary Containment

- Immediate actions to limit damage.
- Use short-term controls (e.g. block traffic, disable accounts, isolate systems).
- Buys time for investigation and proper remediation.



Long-Term Containment

- More permanent changes based on investigation findings.
- Involves fixing root causes, strengthening controls and improving monitoring.
- Becomes part of your overall security improvement plan.

7. Why "Shut Everything Down" Is Not a Good Plan



- Shutting everything down can cause unnecessary business impact and downtime.
- It may destroy valuable evidence and make it harder to understand what happened.
- Attackers may still persist in backups, other systems or cloud environments.
- It can delay a proper and effective response.

A targeted and controlled containment approach is usually more effective than a full shutdown.

8. Smart Containment Steps

1. Identify what is affected.
2. Assess the risk and impact.
3. Contain in a targeted way (isolate, block, disable).
4. Preserve evidence.
5. Monitor for continued malicious activity.
6. Plan and implement long-term containment and remediation.

9. Key Takeaway



- Containment is about stopping or limiting damage, not overreacting.
- Isolate when possible, terminate only when necessary.
- Disable compromised credentials and restrict network access.
- Always preserve evidence.
- Use temporary containment to buy time and follow up with long-term controls.
- Avoid shutting everything down unless it is absolutely necessary.

Eradication and Recovery

1. Remove the Root Cause, Not Only the Symptom



- Identify the real cause of the incident.
- Fix the underlying vulnerability, misconfiguration or weak control.
- Do not just remove visible malicious files and assume the problem is solved.
- Investigate how the attacker gained access and close that path.

Removing the symptom without addressing the root cause allows the attacker to return.

2. Remove Malicious Artifacts



- Remove malicious files, scripts, binaries and persistence mechanisms.
- Check for backdoors, cron jobs, scheduled tasks and unauthorized services.
- Remove unknown users, SSH keys and modified system files.
- Verify that no malicious artifacts remain.

```
# Find and remove suspicious files
$ find / -type f -name "*.sh" -o -name "*.py"
# Check cron jobs
$ crontab -l
$ ls -la /etc/cron.*
# Check for unknown services
$ systemctl list-units --type=service
```

3. Patch Vulnerabilities



- Identify and patch the vulnerabilities that were exploited.
- Update the operating system, applications and dependencies.
- Apply security patches and configuration fixes.
- Verify that the patch has been applied successfully.

```
# Update packages (example: Ubuntu)
$ sudo apt update && sudo apt upgrade
# Check for missing security updates
$ sudo apt list --upgradable
# Verify installed version
$ nginx -v
```

4. Rotate Compromised Credentials



- Change passwords for all affected accounts.
- Rotate API keys, access tokens and SSH keys.
- Revoke active sessions.
- Update credentials in applications, CI/CD, and third-party services.

```
# Expire a user password (Linux)
$ sudo chage -E 0 username
# Rotate cloud access keys (AWS CLI)
$ aws iam create-access-key \
  --user-name username
# List active sessions (Linux)
$ who
```

5. Rebuild When Necessary



- If the system is heavily compromised, rebuild it instead of trying to clean it manually.
- Use a known good image or backup.
- Rebuild containers, VMs or servers with secure baselines.
- Reapply only trusted configurations and data.

Rebuilding is often faster, safer and more reliable than cleaning a heavily compromised system.

6. Restore Trusted Services



- Bring services back online in a controlled and verified manner.
- Ensure configurations are secure.
- Test functionality before full production release.
- Restore data from clean and trusted backups if needed.

```
# Start and check a service
$ sudo systemctl start nginx
$ sudo systemctl status nginx
# Check application health (example)
$ curl -I http://localhost
# Review logs after restart
$ journalctl -u nginx -n 50
```

7. Verify Monitoring



- Ensure logging, monitoring and alerting are working correctly.
- Check that security controls (such as IDS, WAF or threat detection) are enabled.
- Verify that dashboards show the correct data.
- Confirm that alerts will notify the right people.

```
# Check service status (Prometheus example)
$ systemctl status prometheus
# Check recent logs
$ tail -n 50 /var/log/syslog
# Test an alerr (example: send a test alert)
```

8. Watch for Recurrence



- Monitor for signs that the attacker is still active or tries to return.
- Look for repeated failed logins, new suspicious processes or files.
- Check for unexpected network connections or resource creation.
- Keep monitoring for a period after recovery.

```
# Monitor failed logins
$ grep "Failed password" /var/log/auth.log
# Check running processes
$ ps aux | grep -i suspicious
# Monitor network connections
$ ss -tunap
```

9. Contain → Eradicate → Recover → Verify

Contain	Limit the damage and stop further spread.
Eradicate	Remove the root cause and all malicious artifacts.
Recover	Restore trusted services and bring systems back to normal.
Verify	Confirm security, monitoring and stability. Watch for recurrence.

Key Takeaway



- Remove the root cause, not only the symptom.
- Patch, rotate credentials and remove all malicious artifacts.
- Rebuild when necessary and restore trusted services.
- Verify monitoring and watch for recurrence.
- Follow the flow: Contain → Eradicate → Recover → Verify.

Junior DFIR Investigation Lab

Scenario: Suspicious SSH Login on a Linux Server

You receive an alert that an unusual SSH login was detected on a Linux server.
Your task is to investigate, gather evidence and decide if escalation is required.

1. Alert Received

- An alert indicates a successful SSH login from an unusual IP address or at an unusual time.
- Note the alert details: timestamp, username, source IP and server.

```
# Example alert (from monitoring)
Alert: SSH login detected
User: devops
Source IP: 203.0.113.45
Time: 2026-10-15 02:14:22
```

2. Identify Logged-in Users

- Check who is currently logged in to the server.
- Look for unexpected users or multiple sessions.

```
# List current users
$ who
# Detailed login sessions
$ w
# Show all sessions
$ last | head -n 10
```

3. Review Authentication History

- Review successful and failed login attempts.
- Look for repeated failures followed by a successful login.
- Check the source IP addresses and times.

```
# Successful logins
$ last -n 20
# Failed login attempts
$ lastb -n 20
# Check SSH authentication logs
$ sudo grep "sshd" /var/log/auth.log
```

4. Inspect Processes

- Check for suspicious or unexpected processes.
- Look at CPU and memory usage.
- Check the command line used to start the process.

```
# List processes
$ ps aux
# View process tree
$ pstree -p
# Check high resource usage
$ top
```

5. Inspect Network Connections

- Review active network connections.
- Look for unexpected outbound connections.
- Check for unusual listening ports.

```
# Show network connections
$ ss -tulnp
# List connections with processes
$ lsof -i
# Check established connections
$ ss -tpn | grep ESTAB
```

6. Examine Relevant Logs

- Review key system logs for related activity.
- Look for commands, file changes or privilege escalation attempts.

```
# SSH and authentication logs
$ sudo less /var/log/auth.log
# System logs
$ sudo less /var/log/syslog
# Check for sudo activity
$ sudo grep "sudo" /var/log/auth.log
```

7. Build a Short Timeline

- Put the events in chronological order.
- Include the alert, login time, process activity and network connections.

```
# Example timeline (manual)
02:14:22 - SSH login from 203.0.113.45
02:15:10 - New process started (PID 4321)
02:16:05 - Outbound connection to 198.51.100.10
02:17:30 - Suspicious command executed
```

8. Decide Whether Escalation Is Required

- Escalate if you find:
  - Unauthorized access
  - Suspicious processes
  - Data access or exfiltration
  - Privilege escalation
  - Persistence mechanisms

If the activity appears malicious or you are unsure, escalate to the security team.

9. Document Evidence

- Record all findings before taking action.
- Include commands run, their output and timestamps.
- Save relevant log excerpts.
- Document your analysis and recommendation.

```
# Save command output for evidence
$ mkdir -p ~/dfir-evidence
$ ps aux > ~/dfir-evidence/processes.txt
$ ss -tulnp > ~/dfir-evidence/network.txt
$ cp /var/log/auth.log ~/dfir-evidence/
```

Key Takeaway

- Follow a structured, evidence-based investigation process.
- Avoid making changes before collecting and documenting evidence.
- Escalate when there are clear signs of compromise or unauthorized activity.
- Good documentation is critical for further investigation and response.

Incident Response Field Checklist

1. Detect



- Receive an alert or notice unusual activity.
- Understand what triggered the alert.
- Identify the affected system or service.
- Record the time and initial details.

Be alert to logs, monitoring alerts, user reports and abnormal behavior.

2. Verify



- Confirm if the alert is real or a false positive.
- Check multiple sources (logs, metrics, systems).
- Validate the scope and impact.
- Look for obvious signs of malicious activity.

Do not overreact. Verify with evidence.

3. Record



- Write down key details as soon as possible.
- Record timestamps, systems, users and IPs.
- Note what you observed and the tools you used.
- Keep a clear and chronological record.

Accurate records are essential for analysis and reporting.

4. Preserve



- Preserve logs, memory, disk images and evidence before making changes.
- Avoid actions that overwrite or delete evidence.
- Follow your organization's evidence handling process.
- Document what you collect and where it is stored.

Preserve evidence to support investigation and potential legal needs.

5. Investigate



- Examine logs, processes, network connections, users and files.
- Identify what happened, how it happened and what is affected.
- Look for the attack path and root cause.
- Use multiple data sources to build a clear picture.

Follow the evidence, not assumptions.

6. Correlate



- Combine information from different sources.
- Look for patterns, connections and timeline consistency.
- Link related events, users, IP addresses and systems.
- Identify the bigger picture of the incident.

Correlate data to understand the full scope and impact.

7. Escalate



- Escalate if the incident is confirmed, severe or beyond your responsibility.
- Inform the right team (e.g. security, operations, management).
- Provide clear, factual information with evidence.
- Follow your organization's escalation process.

Escalate early when in doubt.

8. Contain



- Limit the attacker's ability to cause further damage.
- Isolate affected systems or restrict network access.
- Disable compromised accounts and credentials.
- Contain in a controlled way to avoid losing evidence.

Contain the threat while preserving evidence.

9. Eradicate



- Remove the root cause, not just the symptom.
- Remove malicious files, users, persistence and backdoors.
- Patch vulnerabilities and fix misconfigurations.
- Rotate compromised credentials.

Ensure the threat is fully removed.

10. Recover



- Restore trusted services and bring systems back to normal.
- Use known good backups or rebuild when necessary.
- Validate that systems are clean and functioning correctly.
- Monitor closely after recovery.

Recover in a controlled and verified way.

11. Verify



- Confirm that the systems are secure and stable.
- Check that monitoring, alerts and security controls are working.
- Look for signs of recurrence or missed activity.
- Validate with multiple sources and tests.

Make sure the incident is fully resolved and not recurring.

12. Document



- Document the full timeline, findings and actions taken.
- Include evidence, decisions and what worked or didn't work.
- Capture lessons learned and recommendations.
- Follow your organization's reporting process.

Good documentation helps prevent future incidents.



Final Junior Engineer Reminder:
Do not guess, follow the evidence.